

Makefile Mastery para Flutter + Supabase

Entiende como funciona un Makefile y por que cada linea del tuyo hace lo que hace. No mas comandos magicos generados por IA.

Lectura completa: ~45 minutos

Parte 1 — Fundamentos de Make

1. Que es Make y por que lo usamos?

1.1 Que es Make?

Make es una herramienta de automatizacion de los anios 70 (si, anterior a internet) que **sigue siendo el estandar** para gestionar proyectos de software. Lee un archivo llamado Makefile y ejecuta comandos segun reglas que tu defines.

La analogia

Makefile = una lista de recetas de cocina

make setup → "Prepara los ingredientes" (instalar dependencias)
make test → "Prueba que la sopa no este salada" (ejecutar tests)
make build-apk → "Cocina el plato final" (compilar APK)

1.2 Por que Makefile y no scripts sueltos?

Aspecto	Scripts sueltos	Makefile
Numero de archivos	10+ scripts	1 archivo
Dependencias	Manual (tu encadenas)	Automatico (target: dependency)
Documentacion	La escribes aparte	make help genera la lista
Variables	Variables de shell	Variables de Make + shell
Solo ejecutar lo necesario	No	Si (solo si cambio la dependencia)
Estandar	Depende del proyecto	Universal en DevOps

1.3 Que hace el Makefile en tu proyecto?

Seccion	Proposito
Desarrollo	Iniciar/Detener Supabase, correr app, regenerar codigo
Calidad	Formatear, analizar, testear
Dependencias	Instalar, actualizar
Supabase	Migraciones, tests de BD, deploy

Build	APK, AAB, web, Linux
Utilidades	Cambiar entorno (.env), validar, limpiar
Git	Conventional commits, hooks

1.4 Make como orquestador entre modulos

Make no solo ejecuta comandos aislados: **conecta todas las herramientas del proyecto en una interfaz unificada.**

Target	Modulo que usa	Que hace
make commit	12-GIT-FLOW	Ejecuta Commitizen (Conventional Commits)
make release	12-GIT-FLOW	Ejecuta standard-version (SemVer + CHANGELOG)
make branch	12-GIT-FLOW	Crea rama con nomenclatura Conventional Branch
make check	Quality gate	format + analyze + test
make ci	11-GITHUB-ACTIONS	Simula pipeline CI local

Ejemplo: hacer un commit

```
# SIN Makefile: tienes que recordar el formato exacto
npx cz

# CON Makefile: un solo comando
make commit
```

Ejemplo: crear una release

```
# SIN Makefile: ejecutar comandos de git manualmente
npm run release
git push --follow-tags

# CON Makefile: un solo comando
make release
```

Ejemplo: crear una rama feature

```
# SIN Makefile: 4 comandos
git checkout develop
git pull
git checkout -b feature/agregar-filtro

# CON Makefile: un solo comando
make branch FEATURE=agregar-filtro
```

Filosofia: Make es la "capa de abstraccion" que conecta Git, Flutter, Supabase y herramientas de calidad en una interfaz unificada.

2. Sintaxis Basica de Makefile

2.1 Estructura de una regla

```
target: prerequisites
      recipe
```

Parte	Que es	Ejemplo
target	El nombre del comando	test:
prerequisites	Lo que debe ejecutarse antes	format analyze
recipe	Lo que realmente hace (indentado con TAB)	flutter test

```
check: format analyze test
      @echo "Todo listo!"
```

2.2 .PHONY — La regla mas importante

```
.PHONY: test clean setup
```

Por que es necesaria? Make asume que cada target es un **archivo**. Si existiera un archivo llamado test en el proyecto, Make diria "test ya esta actualizado" y no ejecutaria nada.

Regla: TODO target que no genere un archivo debe ser .PHONY.

2.3 Variables

```
# Asignacion simple (= evaluacion perezosa)
PROJECT_NAME = mi-proyecto

# Asignacion inmediata (:= evaluacion al momento)
MOBILE_DIR := apps/mobile

# Asignacion condicional (?:= solo si no esta definida)
DEVICE ?= emulator-5554

# Uso
deploy:
    @echo "Desplegando ${PROJECT_NAME}"
    @cd ${MOBILE_DIR} && flutter build apk
```

2.4 El @ — Silenciar comandos

```
test:
    @echo "Ejecutando tests..."
    flutter test
```

Sin @, Make imprime el comando antes de ejecutarlo. Con @, solo se ve la salida del comando.

2.5 Target por defecto

```
.DEFAULT_GOAL := help
```

Cuando ejecutas solo make (sin target), ejecuta help. Asi siempre ves la lista de comandos disponibles.

2.6 Comentarios

```
# Esto es un comentario (linea completa)

clean: ## Limpiar artifacts (comentario inline visible en help)
    rm -rf build/
```

Los comentarios con ## (doble) son usados por el target help para generar la documentacion.

3. Variables Avanzadas y Funciones Shell

3.1 \$(shell ...) — Ejecutar comandos

La funcion mas poderosa: ejecuta un comando de shell y usa su salida como valor.

```
# Detectar Flutter (FVM o global)
FLUTTER := $(shell [ -d .fvm ] && echo fvm flutter || echo flutter)

# Detectar dispositivo Android conectado
DEFAULT_ANDROID = $(shell flutter devices 2>/dev/null \
    | awk -F "•" '/android/ {gsub(/ /, "", $2); print $2; exit}')

# Obtener IP local
LOCAL_IP := $(shell ip -4 addr show \
    | grep -oP '(?<=inet\s)\d+\.\d+\.\d+\.\d+' \
    | grep -v '127.0.0.1' \
    | head -1)
```

3.2 Variables automaticas

Variable	Significado
\$@	Nombre del target actual
\$<	Primer prerequisite
^	Todos los prerequisites
\$?	Prerequisites mas nuevos que el target
\$(@D)	Directorio del target
\$(@F)	Archivo del target

3.3 Condicionales

```
# Ejecutar segun condiciones
run:
    $(if $(DEVICE),\
        flutter run -d $(DEVICE),\
        flutter run\
    )

# Verificar herramientas
deps-check:
    @command -v docker >/dev/null 2>&1 \
        || { echo "Docker no instalado"; exit 1; }
    @command -v supabase >/dev/null 2>&1 \
        || { echo "Supabase CLI no instalado"; exit 1; }
```

3.4 awk y sed en el Makefile

awk -F "•" '/android/ {gsub(/ /,"",\$2); print \$2; exit}'

```
# -F "•"      → separador es " • "
# /android/   → solo lineas con "android"
# gsub(/ /,"") → quita espacios
# print $2    → imprime el segundo campo (la IP/ID)
# exit        → solo el primero
```

sed -i "s|^SUPABASE_URL=.*|SUPABASE_URL=http://\$IP:54321|" .env

```
# s|regex|reemplazo| → substitution
# ^SUPABASE_URL=.*   → linea que empieza con SUPABASE_URL=
# SUPABASE_URL=http://$IP:54321 → nuevo valor
# -i                 → editar in-place (modificar el archivo)
```

3.5 Debugging

```
# Imprimir el valor de una variable (util para depurar)
$(info FLUTTER = $(FLUTTER))
$(warning CUIDADO: dispositivo no encontrado)
$(error ERROR: Flutter no instalado)

# Ejecutar make sin ejecutar (solo muestra lo que haria)
make -n [target]
```

4. Analisis del Makefile Real del Proyecto

Recorrido **linea por linea** de las secciones clave del Makefile del monorepo.

4.1 Variables del proyecto

```
MOBILE_DIR := apps/mobile
WEB_DIR := apps/web
SUPABASE_DIR := supabase
```

Por que? El proyecto es un monorepo (varias apps en un repo). En lugar de escribir apps/mobile cada vez, se usan variables.

4.2 Colores para output

```
BOLD := $(shell tput bold 2>/dev/null)
GREEN := $(shell tput setaf 2 2>/dev/null)
YELLOW := $(shell tput setaf 3 2>/dev/null)
CYAN := $(shell tput setaf 6 2>/dev/null)
RESET := $(shell tput sgr0 2>/dev/null)
```

Por que? Para que la salida de make sea legible visualmente. tput genera codigos de color ANSI.

4.3 Comandos de desarrollo

```
.PHONY: dev-start
dev-start:
    @cd $(SUPABASE_DIR) && supabase start

.PHONY: dev
dev:
    @cd $(MOBILE_DIR) && flutter run --flavor development
```

4.4 Comandos de calidad

```
.PHONY: check
check: format analyze test

.PHONY: format
format:
    @echo "${CYAN}Formateando codigo Dart...${RESET}"
    @cd $(MOBILE_DIR) && dart format .
```

4.5 Storage y Edge Functions

```
.PHONY: deploy-storage
deploy-storage:
    @cd $(SUPABASE_DIR) && supabase db push
    @echo "${GREEN}Storage desplegado (bucket raffa-assets + policies)${RESET}"

.PHONY: deploy-cleanup
deploy-cleanup:
    @cd $(SUPABASE_DIR) && supabase db push
    @cd $(SUPABASE_DIR) && supabase functions deploy raffa-cleanup
```

4.6 Utilidades de entorno

```
.PHONY: env-local
env-local:
    @sed -i "s|^SUPABASE_URL=.*|SUPABASE_URL=http://127.0.0.1:54321|" apps/mobile/.env; \
    echo "${GREEN}SUPABASE_URL=http://127.0.0.1:54321 (modo local)${RESET}"
```

4.7 Conventional Commits (expandido)

Target: make commit

```
.PHONY: commit
commit:
    @npm run commit
```

Flujo completo del target make commit:

```
make commit
├─ 1. npm run commit → ejecuta Commitizen (npx cz)
│   └─ Muestra formulario interactivo:
│       ? Tipo de cambio: [feat/fix/docs/...]
│       ? Alcance (opcional):
│       ? Descripcion:
│       ? Breaking change:
├─ 2. Husky ejecuta pre-commit
│   └─ lint-staged → dart format + flutter analyze
├─ 3. Husky ejecuta commit-msg
│   └─ commitlint → valida formato Conventional Commits
└─ 4. Commit creado: feat(raffles): agregar filtro por fecha
```

Por que no hacer `git commit -m "..."` directamente?

- Humanos olvidan el formato correcto
- El alcance (scope) se pierde
- Los breaking changes no se marcan
- El CHANGELOG no se genera automaticamente

Target: make release

```
.PHONY: release
release: ## Crear release (SemVer + CHANGELOG auto)
    @npm run release
    @git push --follow-tags
    @echo "${GREEN}Release publicado con tags${RESET}"
```

Flujo:

1. standard-version detecta tipo de cambio desde ultimo tag
2. Incrementa version segun SemVer 2.0.0 (feat→minor, fix→patch, breaking→major)
3. Genera/actualiza CHANGELOG.md
4. Crea commit chore(release): vX.Y.Z
5. Crea tag vX.Y.Z
6. Push con tags a origin

4.8 Mapa de dependencias entre targets

```
help          → (autonomo)
setup         → deps → mocks
dev           → check → run
dev-start     → supabase-up → dev
dev-stop      → supabase-down
check         → format → analyze → test
validate      → check ✓
build-apk     → env-prod-check ✓
deploy-storage → supabase db push
deploy-cleanup → deploy-storage + functions deploy
```

Parte 2 — Creacion y CI

5. Creacion Personalizada de Makefiles

5.1 Template minimo

```

# =====
# MAKEFILE - Template minimo para Flutter + Supabase
# =====

PROJECT_NAME := mi-proyecto
MOBILE_DIR := .
SUPABASE_DIR := supabase

SHELL := /bin/bash
.DEFAULT_GOAL := help

.PHONY: help
help:
    @echo "${PROJECT_NAME} - Comandos disponibles"
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
    | sort \
    | awk 'BEGIN {FS = ":.*?## "}; {printf "  %-25s %s\n", $$1, $$2}'

.PHONY: setup
setup: deps mocks ## Instalar dependencias y generar codigo

.PHONY: deps
deps: ## Instalar dependencias
    cd $(MOBILE_DIR) && flutter pub get

.PHONY: mocks
mocks: ## Generar codigo (build_runner)
    cd $(MOBILE_DIR) && dart run build_runner build --delete-conflicting-outputs

.PHONY: dev
dev: check run ## Ejecutar app en modo desarrollo

.PHONY: check
check: format analyze test ## Verificacion completa

.PHONY: format
format: ## Formatear codigo
    cd $(MOBILE_DIR) && dart format .

.PHONY: analyze
analyze: ## Analisis estatico
    cd $(MOBILE_DIR) && flutter analyze

.PHONY: test
test: ## Ejecutar tests
    cd $(MOBILE_DIR) && flutter test

.PHONY: run
run: ## Ejecutar app
    cd $(MOBILE_DIR) && flutter run

.PHONY: clean
clean: ## Limpiar
    cd $(MOBILE_DIR) && flutter clean
    rm -rf $(MOBILE_DIR)/coverage

```

5.2 Targets cross-module (Git Flow + SemVer)

make commit

```
.PHONY: commit
commit: ## Crear commit (Conventional Commits via Commitizen)
@npm run commit
```

make release

```
.PHONY: release
release: ## Crear release (SemVer + CHANGELOG auto)
@npm run release
@git push --follow-tags
@echo "${GREEN}Release publicado con tags${RESET}"
```

make branch

```
.PHONY: branch
branch: ## Crear rama feature (Conventional Branch)
@read -p "Nombre de la feature: " name; \
git checkout develop && \
git pull && \
git checkout -b feature/$$name
@echo "${GREEN}Rama feature/$$name creada desde develop${RESET}"
```

make hotfix

```
.PHONY: hotfix
hotfix: ## Crear rama hotfix
@read -p "Descripcion del fix: " name; \
git checkout main && \
git pull && \
git checkout -b hotfix/$$name
@echo "${GREEN}Rama hotfix/$$name creada desde main${RESET}"
```

make validate

```
.PHONY: validate
validate: check ## Validar codigo completo (format + analyze + test)
@echo "${GREEN}Validacion completa: codigo listo para commit${RESET}"
```

5.3 Buenas practicas

Practica	Por que
cd \$(DIR) && comando	No afecta el shell padre; el cd solo aplica al comando
@echo "mensaje"	Feedback visual sin ruido
command -v herramienta	Verificar existencia de herramienta
{ echo "error"; exit 1; }	Error handling explicito
Variables para directorios	DRY: cambiar en un solo lugar
. PHONY en todos los targets	Evita falsos "up to date"

6. Make en GitHub Actions

Make y GitHub Actions son **complementarios**: Actions orquesta los runners, Make ejecuta la logica del proyecto.

6.1 Por que usar Make en CI?

```
# SIN Make: acciones repetitivas y dificiles de mantener
steps:
  - name: Install deps
    run: cd apps/mobile && flutter pub get
  - name: Format
    run: cd apps/mobile && dart format --output=none --set-exit-if-changed .
  - name: Analyze
    run: cd apps/mobile && flutter analyze
  - name: Test
    run: cd apps/mobile && flutter test

# CON Make: un solo comando
steps:
  - name: Quality check
    run: make check # format + analyze + test en un comando
```

Ventajas:

- **DRY**: la logica esta en el Makefile, no duplicada en YAML
- **Consistencia**: local y CI ejecutan exactamente lo mismo
- **Mantenibilidad**: cambiar la logica en un solo lugar

6.2 Workflow CI tipico con Make

```

name: CI
on: [push, pull_request]

jobs:
  quality:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: apps/mobile

    steps:
      - uses: actions/checkout@v4

      - uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.24.0'

      - name: Cache pub
        uses: actions/cache@v4
        with:
          path: ~/.pub-cache
          key: ${{ runner.os }}-flutter-`${{ hashFiles('apps/mobile/pubspec.lock') }}`

      - name: Setup
        run: make setup

      - name: Quality gate
        run: make check

      - name: Coverage
        run: make coverage

```

6.3 Comandos de Make usados en CI/CD

Comando	En CI hace
make setup	flutter pub get + build_runner
make check	format + analyze + test
make coverage	Tests con reporte de cobertura
make build-apk	Compila APK release
make supabase-up	Inicia Supabase para tests
make db-push	Aplica migraciones en produccion

6.4 Make con condiciones en CI

```

# El Makefile puede adaptarse segun el entorno
ifdef CI
  FLUTTER := flutter # en CI no hay FVM
  EXTRA_FLAGS := --no-sound-null-safety
else
  FLUTTER := $(shell [ -d .fvm ] && echo fvm flutter || echo flutter)
endif

test:
  $(FLUTTER) test $(EXTRA_FLAGS)

```

```
# En GitHub Actions:
- name: Test
  run: make test
  env:
    CI: true
```

6.5 Patron: Make + matrix builds

```
strategy:
  matrix:
    flutter_version: ['3.22.0', '3.24.0']

steps:
- uses: subosito/flutter-action@v2
  with:
    flutter-version: ${ matrix.flutter_version }

- run: make check
```

7. Ejercicios de Makefile

Ejercicio 1: Leer y entender

```
# Dado este fragmento del Makefile real:
.PHONY: env-ip
env-ip:
    @echo "${CYAN}Detectando IP local...${RESET}"
    @IP=$(ip -4 addr show | grep -oP '(?<=inet\s)\d+\.\d+\.\d+\.\d+' | grep -v '127.0.0.1' | head -1);
    \
    if [ -z "$$IP" ]; then \
        echo "${YELLOW}No se pudo detectar IP automaticamente${RESET}"; \
        exit 1; \
    fi; \
    sed -i "s|^SUPABASE_URL=.*|SUPABASE_URL=http://$$IP:54321|" apps/mobile/.env; \
    echo "${GREEN}SUPABASE_URL=http://$$IP:54321 (modo WiFi)${RESET}"
```

Preguntas:

1. Que hace `grep -v '127.0.0.1'`?
2. Que pasa si `head -1` no encuentra ninguna IP?
3. Por que usa `$$IP` y no `$IP`?

Ejercicio 2: Depurar un target

```
# Este target tiene un error. Cual?
.PHONY: test-all
test-all: test-mobile test-web

.PHONY: test-mobile
test-mobile:
    cd apps/mobile
    flutter test

.PHONY: test-web
test-web:
    cd apps/web
    npm test
```

Solucion: El error esta en la indentacion de test-mobile y test-web: la receta esta separada en dos lineas. En Make, cada linea de la receta se ejecuta en un subshell diferente. El `cd apps/mobile` se ejecuta en su propio shell y no afecta al `flutter test`.

```
test-mobile:
    cd apps/mobile && flutter test

# O mejor, usar working-directory en CI o variables
MOBILE_DIR := apps/mobile
test-mobile:
    cd $(MOBILE_DIR) && flutter test
```

Ejercicio 3: Anadir un nuevo target

```
# Anade un target "seed" que ejecute el seed.sql de Supabase:
# supabase db reset (para limpiar datos)
# psql -f supabase/seed.sql (para cargar datos de prueba)

.PHONY: seed
seed: db-reset ## Resetear BD y cargar datos de prueba
    @echo "Cargando datos de prueba..."
    @cd $(SUPABASE_DIR) && psql -f seed.sql
    @echo "${GREEN}Datos de prueba cargados${RESET}"
```

Ejercicio 4: Target con variable condicional

```
# Crea un target "run" que:
# - Si se define DEVICE, ejecuta: flutter run -d $(DEVICE)
# - Si no, ejecuta: flutter run
# - Si DEVICE es "web", ejecuta: flutter run -d chrome

.PHONY: run
run: ## Ejecutar app (usa DEVICE si esta definido)
    @if [ "$(DEVICE)" = "web" ]; then \
        cd $(MOBILE_DIR) && flutter run -d chrome; \
    elif [ -n "$(DEVICE)" ]; then \
        cd $(MOBILE_DIR) && flutter run -d $(DEVICE); \
    else \
        cd $(MOBILE_DIR) && flutter run; \
    fi
```

Ejercicio 5: Crear un Makefile desde cero

```

PROJECT_NAME := mi-app
.DEFAULT_GOAL := help

.PHONY: help
help:
    @echo "${PROJECT_NAME} - Comandos"
    @grep -E '^[a-zA-Z_-]+:.*?## .*$$' $(MAKEFILE_LIST) \
    | sort | awk 'BEGIN {FS = ":.*?## "}; {printf " %-20s %s\n", $$1, $$2}'

.PHONY: setup
setup: deps mocks ## Setup completo

.PHONY: deps
deps: ## Instalar dependencias
    flutter pub get

.PHONY: mocks
mocks: ## Generar codigo
    dart run build_runner build --delete-conflicting-outputs

.PHONY: dev
dev: check run ## Modo desarrollo

.PHONY: check
check: format analyze test ## Quality gate

.PHONY: format
format: ## Formatear
    dart format .

.PHONY: analyze
analyze: ## Analisis
    flutter analyze

.PHONY: test
test: ## Tests
    flutter test

.PHONY: run
run: ## Ejecutar
    flutter run

.PHONY: build-apk
build-apk: ## Build APK
    flutter build apk --release

.PHONY: clean
clean: ## Limpiar
    flutter clean
    rm -rf coverage/

```

Ejercicio 6: Targets cross-module (Git Flow + SemVer)

Crea los siguientes targets en tu Makefile. Cada uno conecta Make con herramientas del modulo 12 (Git Flow + Conventional Commits + SemVer).

6.1 Target make commit

```

.PHONY: commit
commit: ## Crear commit (Conventional Commits via Commitizen)
    @npm run commit

```

Preguntas:

1. Que pasa si el usuario escribe un commit que no cumple el formato?
2. Que papel juega Husky en este target?

6.2 Target make release

```
.PHONY: release
release: ## Crear release (SemVer + CHANGELOG auto)
    @npm run release
    @git push --follow-tags
    @echo "${GREEN}Release publicado con tags${RESET}"
```

Preguntas:

1. Como determina standard-version si debe incrementar minor, patch o major?
2. Que archivos genera standard-version?
3. Por que --follow-tags en el push?

6.3 Target make branch

```
.PHONY: branch
branch: ## Crear rama feature (Conventional Branch)
    @read -p "Nombre de la feature: " name; \
    git checkout develop && \
    git pull && \
    git checkout -b feature/$$name
    @echo "${GREEN}Rama feature/$$name creada desde develop${RESET}"
```

Preguntas:

1. Por que se usa \$\$name en lugar de \$name?
2. Que pasaria si no se ejecuta git pull antes de crear la rama?

6.4 Target make hotfix

```
.PHONY: hotfix
hotfix: ## Crear rama hotfix
    @read -p "Descripcion del fix: " name; \
    git checkout main && \
    git pull && \
    git checkout -b hotfix/$$name
    @echo "${GREEN}Rama hotfix/$$name creada desde main${RESET}"
```

Preguntas:

1. Por que hotfix se crea desde main y no desde develop?
2. Cual es la diferencia entre make branch y make hotfix?

6.5 Ejercicio completo: Integrar todo

```
# Verificar que Commitizen esta instalado
check-tools:
    @command -v npx >/dev/null 2>&1 || { echo "npx no encontrado"; exit 1; }
    @test -f package.json || { echo "package.json no encontrado"; exit 1; }
    @echo "${GREEN}Herramientas cross-module listas${RESET}"

# Flujo completo: validate → commit
full-commit: validate commit ## Validar código y crear commit
    @echo "${GREEN}Commit creado con validación completa${RESET}"
```

Checklist

- Entiendo la sintaxis básica: target, prerequisites, recipe
- Se que es .PHONY y por que es necesario
- Puedo leer `$(shell ...)` y se que hace
- Entiendo awk y sed cuando aparecen en un Makefile
- Se depurar con `make -n $(info ...)`
- Puedo añadir un nuevo target
- Puedo crear un Makefile desde cero
- Entiendo como Make se integra con GitHub Actions
- Puedo crear targets cross-module (commit, release, branch, hotfix)